Lecture 17 - Storage and LSM trees

Today's goals:

- Wrap up C-Store and other contemporary systems
- Introduce LSM Trees

Broader Context for C-Store

- Recall the N-ary Storage Model (NSM)
  - Traditional storage: entire tuple stored together
- C-Store instead uses a form of Decomposition Storage Model (DSM)
  - The fewer columns you read, the better the relative perf of DSM over NSM
  - C-Store introduced a number of innovations beyond DSM (which came earlier)
  - e.g., compression, column projections, etc.
- I want to chart the trajectory of MonetDB, a different, older DSM

MonetDB

- Developed at CWI (Amsterdam) starting in the early 1990s
- Pioneered by Martin Kersten, Peter Boncz (then a grad student)
- Physical data model: each column stored as a **BAT (Binary Association Table)**
  - A BAT is a two-column table: `(OID, value)`
    - `OID` is typically omitted given it may be implicit (e.g., based on position)
  - Algebraically simple: every operator takes one or two BATs and produces a BAT
    - Query Language Paper: MIL primitives for querying a fragmented world, VLDBJ 1999
  - Example:
    - Suppose we have table: `(name, age, salary)`
    - Three BATs `(oid, name), (oid, age), (oid, salary)`
    - Operator example:
      - `candidates = select (BAT_age, "> 25")
        - output schema: `(oid_new, val_new)` ->
          - here `val_new` will be matching `oids` for those with age > 25
          - and `oid_new` will be positional
      - `result = project (candidates, BAT_name)`
        - output schema: `(oid_new2, val_new2)` ->
          - here `val_new2` will be populated with names from looking up `candidates[2]` values in `BAT_name[1]`, and returning `BAT_name[2]`
      - Q: Interestingly, a `join` creates two output BATs - why?
        - `(oid, lhs_oid), (oid, rhs_oid)` -
        - need to remember which left tuple joined with which right tuple
        - so that subsequent `project` calls can stitch results together correctly.
- Query execution: unlike volcano-based iterator model, instead used column-at-a-time bulk processing
  - One operator processes an *entire column* before passing results to the next operator
  - Example:
    - `SELECT` on a 100M-row column reads all 100M values,
    - produces a 100M-element result BAT, passes that to the next stage
  - Downsides: no pipelining, intermediate results can be very large
  - Upsides:

        - cache-friendly operation on a sequence of values, no function overhead per tuple
- In the 90s, made sense: main-memory was still small, reading individual columns at a time and batching writes out to disk was reasonable

Vectorwise aka MonetDB/X100

- Designed to address MonetDB's full materialization problem while preserving CPU efficiency, led by Marcin Zukowski (who later founded Snowflake) and Peter Boncz
- Key paper: MonetDB/X100: Hyper-Pipelining Query Execution, CIDR'05
- Idea: **vector-at-a-time** (vectorized) execution
    - Instead of tuple-at-a-time (Volcano) or column-at-a-time (MonetDB), process a **small vector** (e.g., 1000–10000 values) at a time
    - Each operator is a tight loop over a fixed-size array — fits in L1/L2 cache
    - Operators can still be pipelined:
        - when a vector is done, it flows to the next operator (like Volcano but with vectors not single tuples)
- Benefits vs. tuple-at-a-time (Volcano):
    - No per-tuple function call overhead
    - SIMD instructions can be applied to vector — hardware can process multiple values in one CPU instruction
    - Excellent branch prediction — repeated tight loops over homogeneous data
- Benefits vs. column-at-a-time (MonetDB):
    - Intermediate results are at most `vector_size` rows — fit in cache, no large materializations
- VectorWise spun off in 2008 and acquired by Ingres(!) in 2011, later renamed to Actian Vector.
- The ideas live on most prominently in DuckDB and Snowflake, both led by authors of Vectorwise

Database Cracking

- Idreos, Kersten, and Manegold, *"Database Cracking"*, CIDR 2007. Paper
    - Won test of time award just this year!
- Index construction/maintenance during query processing
- Each query interpreted as advice to "crack" the physical database store into smaller pieces.
- Motivation: in dynamic environments with no idle time for upfront index construction and no a-priori workload knowledge, traditional offline index building is impractical
- Core mechanism — selection cracking:
    - When a query requests `A < 10`, the system physically *rearranges* the column in place
    - so that all values with `A < 10` are clustered at the front and all `A ≥ 10` at the back
        - a partial partition, like one pass of quicksort's partition step
    - The boundary positions recorded in a *cracker index* — a lightweight B-tree
    - A subsequent query requesting `A > v` where `v ≥ 10` only needs to search the already-separated suffix
    - A third query with a finer predicate only needs to re-crack the pieces that straddle its boundaries, leaving already-clean pieces untouched
    - The more often a key range is queried, the more refined and sorted its representation becomes
        - aka *self-organizing*

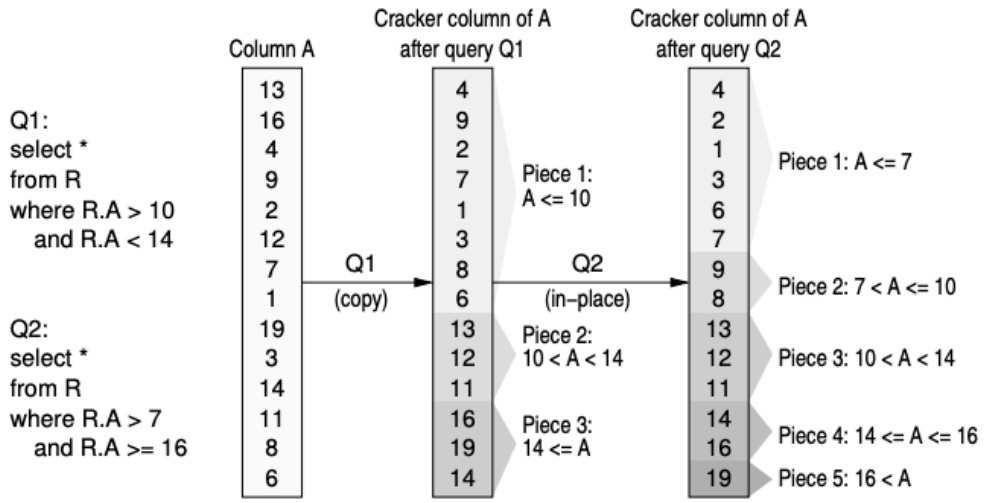


Figure 1: Cracking a column

- Comparison to traditional indexing:
  - B-tree: pay full index build cost upfront, amortized over all future queries
    - Full sort: even faster per-query lookups, but even higher upfront cost
    - Cracking: zero upfront cost; the first query is essentially a scan,
      - but each subsequent query incrementally improves the physical layout with no background work
    - Non-queried columns remain entirely unindexed — no wasted effort
  - Downsides:
    - Sensitive to workload pattern.
    - With adversarial or highly skewed queries (e.g., always cracking at the same boundary), performance degrades.
- Built on top of MonetDB; tightly coupled to the BAT column model
- Essentially, a lazy indexing scheme where column data is physically re-arranged incrementally as queries arrive, with no separate index build phase.
- Not all that widely adopted, but widely known in the research literature

PAX (Partition Attributes Across) Storage Model

- A different stream of work, led by Anastasia Ailamaki (now at EPFL) as a grad student with David DeWitt, Wisconsin
  - Canonical paper Weaving Relations for Cache Performance, VLDB 2001
- Unlike DSM, PAX retains compatibility with NSM, but aims to improve **in-page cache performance**
  - So, leave the page boundaries alone, but reorganize interior of each page so that values are grouped by attribute.
  - Essentially mini-columns within a page
  - When an operator loads a page into the buffer pool, the values for `age` are contiguous in cache,
    - the values for `salary` are contiguous in cache, and tuple reconstruction is cheap (everything is on one page).
- PAX is completely transparent to the storage manager; purely a cache optimization.
- Downside: it doesn't reduce *disk* I/O when you have many columns and large relations. That still requires true column storage.

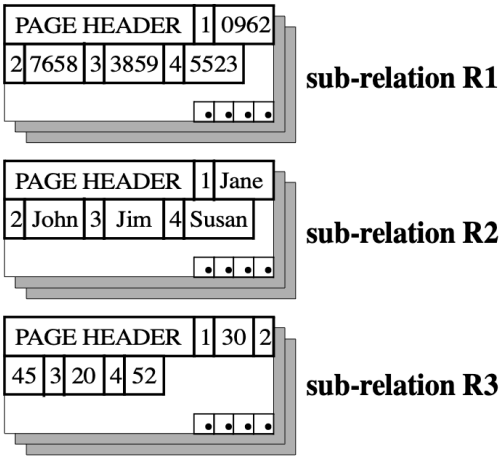


FIGURE 2: *The Decomposition Storage Model (DSM).*

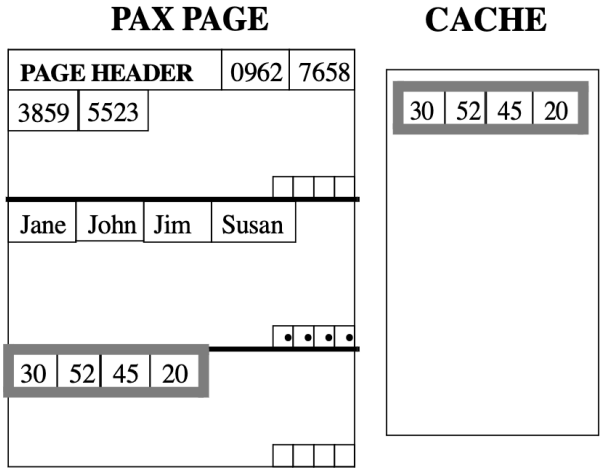


FIGURE 3: *The cache behavior of PAX.*

-
- Interesting note: Snowflake's internal page format is PAX-style:
  - it uses row groups (horizontal partitions), and within each row group data is stored column-by-column. This gives both columnar compression/pruning and locality for tuple reconstruction.

More on the C-Store Lineage

- Integrating Compression and Execution in Column-Oriented Database Systems, SIGMOD 2006
  - showed operating directly on compressed data (without decompression) is a major win
- Another important paper in the C-Store lineage studied when to reconstruct tuples Materialization Strategies in a Column-Oriented DBMS, ICDE'2007 - let's discuss briefly
- Standard approach: early materialization
  - After reading the needed columns, stitch them together into rows (partial tuples) as early as possible
  - Then apply predicates and further operators on rows
  - Pros: looks like a row store internally after the initial scan — can reuse existing operators
  - Cons: once columns are stitched, all subsequent operators must read and process more data per "record" (all the extra columns even if not needed)
- Alternative: late materialization
  - Keep each column separate for as long as possible
  - Produce a bitstring (position list / bitmap) identifying which rows survive each predicate
  - Only stitch columns together into rows at the very end — or not at all if the query only needs aggregates
  - BAT can also achieve this but figuring out when to do this is non-trivial
  - Pros:
    1. *Fewer tuples constructed*: if a predicate eliminates 95% of rows, avoid constructing those rows entirely
    2. *Compressed-domain evaluation*: with late materialization, predicates can operate directly on compressed columns
    3. *Cache efficiency/SIMD-friendly*: operating on a single column array keeps related data contiguous — fits better in cache than operating on wide "partial rows", and same story for SIMD
  - Cons: more complex query executor; must track position lists (bitstrings) carefully through the pipeline
- Takeaway: materialize early if nearly all rows survive — late materialization carries the overhead of position list management without the payoff

HyPer

- From Neumann, "Efficiently Compiling Efficient Query Plans for Modern Hardware" (VLDB 2011)
- HyPer/Umbra compiles entire pipelines to machine code via LLVM

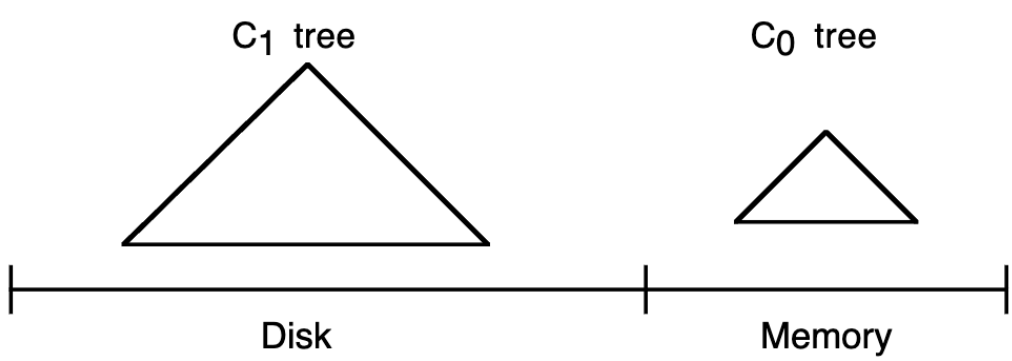
- Core idea: query compilation to machine code
  - Volcano iterator model has high per-tuple overhead
  - HyPer compiles entire query pipelines directly to LLVM IR, then JIT-compiled to native machine code
- Downside: compilation latency that makes HyPer less suitable for short queries.
- **Umbra** is Neumann's successor to HyPer

Analytical Takeaways

- Several takeaways:
  - Operating on columns as small vectors, taking advantage of vectorization, SIMD, and compression — and pipelining,
  - Pipelining those vectors across operators
  - Combining with late materialization
  - The RS/WS split (C-Store) or equivalent immutability tricks (SSTables) to enable compression and avoid locking
- is a good idea - very popular across analytical systems
- No amount of indexes, MVs, joins, and query optimizations help overcome storage issues
- Modern descendants: Vertica (from C-Store), Amazon Redshift, BigQuery (from Dremel), Snowflake (columnar + PAX-style pages), DuckDB (vectorized, in-process), Apache Parquet (columnar storage format), Apache Arrow (in-memory columnar format)

LSM Trees

- O'Neil, Cheng, Gawlick, and O'Neil, *"The Log-Structured Merge-Tree (LSM-Tree)*", Acta Informatica, 1996
  - A difficult read! But very influential!
- Goal: **low-cost indexing when there is a high rate of updates**
- Do so by reducing the amount of random I/O
  - deferring and batching index changes
- Unlike B+trees which are good for reads, but do poorly for writes
- As the main con, LSM trees are not very good for reads
- So main application: reads/retrieval is much less frequent than insert
- Basic idea: have a hierarchical index, split between memory and disk
- Warmup: two components



  - With new inserts:
    - Add it to WAL
    - Add it to C0 tree in memory (but no need to add it to C1)
    - So this incurs no random I/O cost
  - Search: first look in C0 then in C1
  - **Rolling merge (C0 → C1):**
    - when C0 reaches a threshold size, initiate an ongoing rolling merge:

        - take a contiguous portion of C0, merge-sort it with the corresponding portion of C1, write out the result as new C1 pages
        - This is sequential I/O on both input and output - so good
        - Old C1 pages and the merged C0 entries are deleted
    - Can extend to multiple components (C0 → C1 → ... → Cn), each a B+ tree and each one order of magnitude larger than the previous
- Extended and operationalized first by the BigTable paper from Google
    - *Bigtable: A Distributed Storage System for Structured Data* (Chang et al., Google, OSDI 2006)
    - An influential K-V store
        - Influenced Cassandra, HBase
        - Open-sourced as LevelDB in 2011, Open-source RocksDB (from Facebook) builds on this
    - **Not just for indexing, but for all storage!**
        - the LSM becomes the entire storage engine
    - We have memtables and SStables
    - In-memory table: *memtable*
        - Skip list or a balanced BST is also OK
            - Q: what is a skip list?
                - Idea:
                - Start with a sorted linked list. Search is O(n)
                - Skip list maintains *multiple layers* of linked lists stacked on top of each other
                - The bottom layer contains all elements in sorted order,
                - each layer above is an "express lane" that skips over more and more elements
                - Search: start at top left and move right as long as next element is ≤ your target, then drop down a level and repeat.
                - Analogous to how a binary search works, but over a linked structure rather than an array.
                - Insertions: probabilistic. Insert into bottom most layer, with probability p add it to next year and so on
        - When the memtable fills up (configurable threshold, often 64–256 MB), frozen and written to disk as an SSTable;
            - a new empty memtable is created for incoming writes.
            - The frozen memtable is kept as an *immutable memtable* briefly until the flush completes.
    - On-disk: *SSTables* (sorted sequence/string tables)
        - Immutable sorted files, making concurrency, recovery, compaction easy
        - From Optimizing Space Amplification in RocksDB, CIDR'17, S

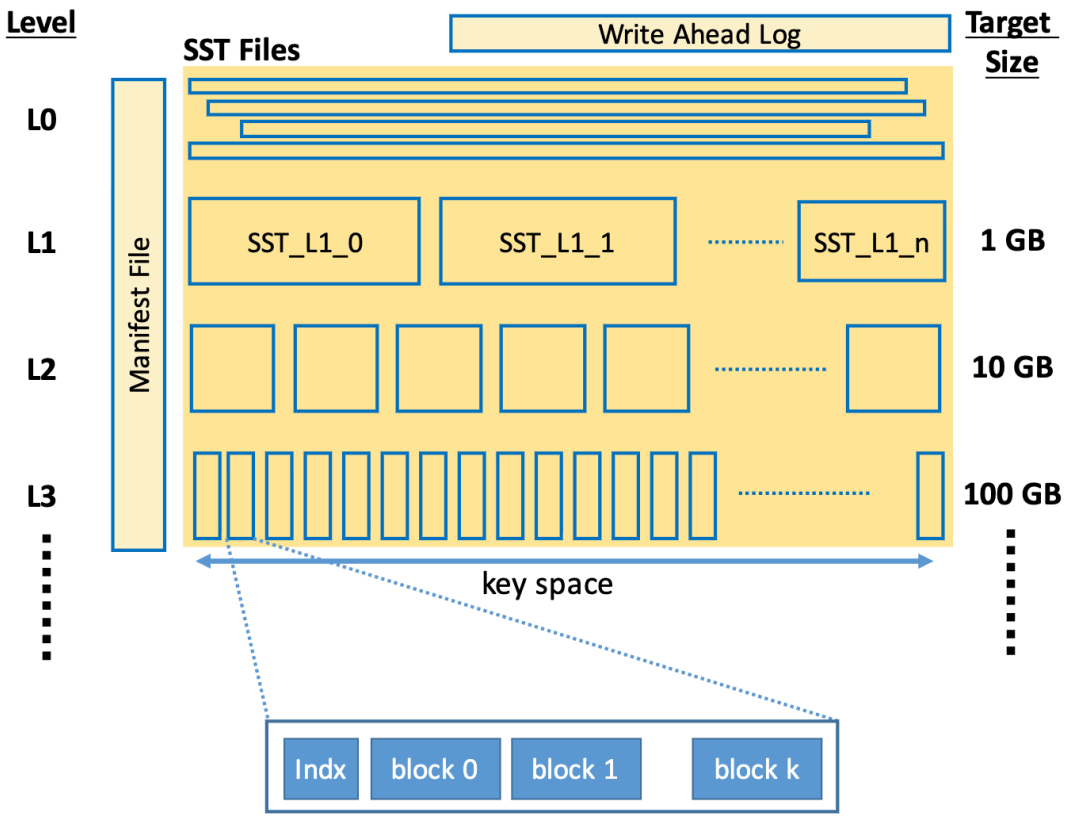


Figure 1: SST file organization. Each level maintains a set of SST files. Each SST file consists of unaligned 16KB blocks with an index block identifying the other blocks within the SST. Level 0 is treated differently in that its SST files have overlapping key ranges, while the SST files at the other level have non-overlapping key ranges. A *manifest file* maintains a list of all SST files and their key ranges to assist lookups.

- SSTables have levels L0 ... Ln, each one order of magnitude larger than the previous.
- **L0 is special:** SSTables at L0 may have overlapping key ranges
  - They are flushed directly from the memtable without regard to what already exists at L0.
  - All other levels have non-overlapping key ranges within the level
- Each SSTable has:
  - a sequence of data *blocks* (default 4 KB each),
  - followed by an *index block* (one key per data block — a sparse index),
  - followed by metadata (bloom filter, stats)
- *Fence pointers:* the index block within each SSTable stores (min_key, max_key, block_offset) triplets for each data block.

- A **manifest** file records which SSTables exist at each level and their key ranges
  - source of truth for the LSM tree's structure
- Compaction process:
  - For L0, we keep count of number of SST and initiate compaction if the count grows beyond a certain number
  - "*For L>0, when the combined size of all SSTs in level-L exceeds a threshold (e.g.,* $10^{(L-1)}$*GB) then one or more level-L SSTs are selected and merged with the overlapping SSTs in level(L+1), after which the merged level-L and level-(L+1) SSTs are removed.*"

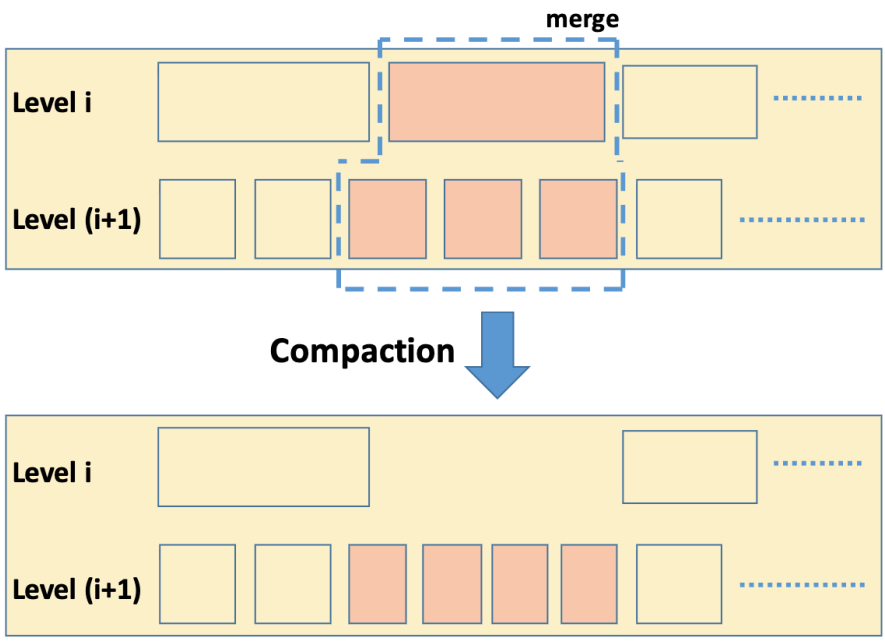


Figure 2: Compaction. The contents of a selected level-i SST file is merged with those SST files at level i+1 that have key ranges overlapping with the key range of the level-i SST. The shaded SST files in the top part of the figure are deleted after the merge process. The shaded SST files in the bottom part of the figure are new files created from the compaction process. The compaction process removes data that has become obsolete; i.e., data that has been marked as deleted and data that has been overwritten (if they are no longer needed for snapshots).

- Compaction gradually migrates new updates from Level-0 to the last level
- Essentially a merge-sort, since both levels are sorted
  - Compaction also handles deletes: deleted keys are marked with *tombstones*
    - special sentinel entries that suppress any older version of the same key.
    - Tombstones are only physically removed when they are compacted past all older levels that might contain the deleted key.
  - *Tiered compaction* (alternative policy):
    - instead of merging one file into the next level,
    - accumulate multiple similarly-sized SSTables at a level and then merge them all at once.
    - Lower write amplification but higher read amplification and space amplification.
- Retrieval:
  - Can also use bloom filters to reduce lookups
  - Can also use what are called *fence pointers*
  - Check memtable first (in-memory, fast)
  - Then check each level from L0 downward:
    - At L0: must check *all* SSTables (overlapping ranges) — use bloom filter first to skip files that definitely don't contain the key
    - At L>0: use manifest to identify the one SSTable whose key range covers the target; then within that SSTable, three binary searches:
      - (1) locate the SSTable via manifest,
      - (1) locate the right data block via the index/fence pointers,

        - (3) binary search within the data block
- Narrative around amplification:
    - Space amplification - less storage than B+trees because B+trees are only 1/2 to 2/3 full, while every block here can be dense-packed, so no fragmentation
    - Read amplification - relative to B+trees this is worse, since we have to read a lot more, across all levels, and 3 searches per level
    - Write amplification - less than B+trees, because each write can be just done to memtable, rest can be done "in background"
- Modern KV stores using LSM storage engines
    - NoSQL databases including Cassandra, BigTable, HBase, LevelDB, RocksDB, ElasticSearch
    - RocksDB is used by CockroachDB and Kafka/Flink
    - Also Salesforce - CIDR'26 a nice description of modern LSM tree concerns